

RSA 부분 소수 비트 유출 문제

작업 에셋 및 재현 문서

2026 암호분석경진대회 7 번 문제
외부 실행 환경에서 이어서 돌리기 위한 상세 정리

요약: 이 문서는 현재까지 수행한 문제 분석, 격자/SAT 공격 시도, 남은 실행 계획, 패키지 사용법을 정리한다. 현재 패키지는 완성된 풀이가 아니라 외부 Sage/cuso/flatter 환경에서 계속 실행하기 위한 작업 에셋이다. 최종 p, q 및 평문은 아직 복구되지 않았다.

1. 문제 요약

입력 데이터. 문제 PDF 1 쪽에는 2048 비트 RSA 모듈러스 $N = p \cdot q$, 공개지수 $e = 65537$, 암호문 ct, 그리고 1024 비트 소수 p 에 대한 부분 비트 누출 정보 MASK 와 p & MASK 가 주어진다. MASK 에서 1 인 위치가 p 의 알려진 비트이다.

목표. 누출된 p 의 비트를 이용해 N 을 인수분해하고 textbook RSA $ct = m^e \bmod N$ 을 복호화하여 평문 m 의 big-endian 바이트열을 출력한다.

2. 비트 구조 분석

MASK 를 LSB 기준으로 스캔한 결과는 다음과 같다.

구분	값
p 전체 크기	1024 비트
N 크기	2048 비트
알려진 p 비트	672 비트
미지 p 비트	352 비트
알려진 비율	65.625%

미지 비트 블록은 다음 7 개이다.

LSB 기준 비트 범위	길이	비고
150..153	4	low nibble brute force
265..348	84	x 그룹 내부
362..419	58	x 그룹 내부
600..668	69	y 그룹 내부
682..768	87	y 그룹 내부
784..829	46	y 그룹 내부
920..923	4	high nibble brute force

핵심 단순화는 양끝의 4 비트 블록 두 개만 브루트포스한 뒤, 나머지 미지 영역을 두 개의 큰 변수로 묶는 것이다.

```
low = p[150..153] # 4-bit candidate
high = p[920..923] # 4-bit candidate
x = p[265..419] # 155-bit grouped variable
y = p[600..829] # 230-bit grouped variable
p = p0 + 2^265*x + 2^600*y
```

주의할 점은 x 와 y 내부에 원래는 이미 알려진 gap 비트가 포함된다는 것이다. 따라서 small-root 후보를 얻은 뒤에는 항상 다음 조건으로 검증한다.

```
N % p == 0
(p & MASK) == LEAK
```

3. 공격 모델과 이론적 근거

이 문제는 Coppersmith 계열의 부분 키 노출 공격에 해당한다. 단일 연속 미지 블록이면 univariate Coppersmith 를 직접 적용할 수 있지만, 이 문제는 알려진 비트가 여러 위치에 있으므로 Herrmann-May 식 “known composite N 의 unknown divisor p 에 대한 선형식 small root” 모델이 적합하다.

$$f(x, y) = p_0 + 2^{265}x + 2^{600}y$$

$$f(x, y) \equiv 0 \pmod{p}, \text{ where } p \mid N$$

두 변수의 bound 곱은 다음과 같다.

$$X * Y = 2^{155} * 2^{230} = 2^{385} = N^{(385/2048)} \approx N^{0.188}$$

Herrmann-May 의 arbitrary-bit factoring 결과는 unknown bit block 수가 커질수록 실행 시간이 커진다는 점을 시사한다. 따라서 7 변수/5 변수보다 2 변수로 묶는 전략이 더 현실적이다. 최신 자동화 도구인 cuso 는 multivariate Coppersmith shift polynomial 선택을 자동화하고, modulus="p"와 modulus_multiple=N 형태를 직접 지원하므로 이 문제의 우선 실행 경로로 적합하다.

4. 지금까지 시도한 구현

4.1 마스크 파싱 및 구조 확인

문제 상수를 파싱하여 N 의 bit length, MASK 의 bit count, LEAK 의 mask 일관성을 확인했다. 재현 스크립트는 src/solve7_main.py --mode analyze 및 src/inspect_problem.py 이다.

```
$ python3 src/solve7_main.py --mode analyze
N bits = 2048
known p bits = 672 / 1024
unknown p bits = 352 / 1024
...
```

4.2 SAT 및 bit multiplication 인코딩

초기 접근은 p 의 알려진 비트를 Boolean 상수로 고정하고 $p \cdot q = N$ 곱셈 회로를 CNF/XOR 로 인코딩하는 것이었다. 전체 곱셈 인코딩은 매우 커졌고, 이후 low/high 4 비트 후보를 고정하여 $q \pmod{2^{265}}$ 와 q 의 상위 prefix 를 유도하는 축소 SAT 도 만들었다. 그래도 후보 하나당 100 만 변수급 문제가 되어 현재 단일 컨테이너에서는 부담이 컸다.

파일	용도
sat_factor.py	초기 전체 곱셈 SAT 인코딩
sat_candidate_opt.py	low/high 후보 고정 축소 인코딩
sat_mul.py, sat_mul_cnf.py	XOR-native 및 CNF 곱셈 회로 생성
run_sat_one.py, run_sat_cnf_one.py	후보 1 개 실행용 wrapper
z3_factor.py, z3_xy.py	Z3 기반 sanity experiment

4.3 5 변수 Herrmann-May

low/high 4 비트를 고정한 뒤 남은 다섯 블록을 각각 변수로 두는 형태도 구현했다.

$$p = p_0 + 2^{265}x_1 + 2^{362}x_2 + 2^{600}x_3 + 2^{682}x_4 + 2^{784}x_5$$

$m=3$, $t=2$ 는 56 차원 정도로 실행되었으나 충분히 강한 보조 다항식이 나오지 않았다. $m=4$, $t=2$ 이상은 fpylll 환경에서 오래 걸렸다.

4.4 2 변수 Herrmann-May 및 루트 복원

현재 가장 유망한 방향이다. 2 변수 격자에서 LLL 로 얻은 보조 다항식의 공통근을 작은 소수체 위에서 찾고 CRT 로 합치는 방식, resultant/Groebner 방식 등을 실험했다. 현재 컨테이너에는 Sage/cuso/flatter 가 없어 cuso 실행은 하지 못했고, pure fpylll fallback 은 일부 파라미터에서 p 를 찾지 못했다.

시도	결과
cuso mode	컨테이너에 Sage/cuso/flatter 없음. 외부 실행 권장.
local m=7,t=2 cid=0	LLL 실행됨. p 미발견.
local m=8,t=3 cid=0	LLL 실행됨. p 미발견.
5 변수 m=4 계열	차원/성능 병목. 외부 고성능 LLL 필요.

5. 패키지 사용법

패키지 루트에서 다음 명령으로 구조를 확인한다.

```
sage -python src/solve7_main.py --mode analyze
```

권장 경로는 cuso 전체 후보 실행이다.

```
sage -python src/solve7_main.py --mode cuso --a 0 --b 256 | tee logs/cuso_full.log
```

후보 id 는 $cid = 16 * high + low$ 이다. 범위를 나누려면 다음처럼 실행한다.

```
bash scripts/run_cuso_range.sh 0 64
bash scripts/run_cuso_range.sh 64 128
bash scripts/run_cuso_range.sh 128 192
bash scripts/run_cuso_range.sh 192 256
```

8 개 병렬 작업은 다음 스크립트로 실행한다.

```
bash scripts/run_cuso_8way.sh
```

fallback 모드는 다음과 같다. 이 경로는 재현성과 sanity check 용이며 최종 해결 경로로는 약하다.

```
python3 -m pip install -r env/requirements-local.txt
python3 src/solve7_main.py --mode local --m 8 --t 3 --lead y --a 0 --b 1
```

6. p 가 나온 뒤 복호화

solve7_main.py 는 p 후보가 검증되면 자동으로 q 를 계산하고 복호화한다. 외부 도구로 p 만 얻은 경우에는 다음을 실행한다.

```
python3 src/decrypt_with_factor.py <p-hex-or-decimal>
```

내부 복호화 코드는 표준 RSA 절차이다.

```
q = N // p
phi = (p - 1) * (q - 1)
d = pow(65537, -1, phi)
m = pow(ct, d, N)
plaintext = m.to_bytes((m.bit_length() + 7) // 8, "big")
```

7. 패키지 구성

경로	내용
problem/	원본 문제 PDF
src/	압축 solver, bit 분석, 복호화 helper
scripts/	외부 실행 wrapper
env/	설치 안내, fallback requirements, Dockerfile.local
docs/	이 문서와 Markdown 버전
experiments/	중간 실험 코드, 로그, basis 파일
logs/	실행 로그 저장 위치

8. 남은 작업 우선순위

1. SageMath 9.8+, cuso, flatter, msolve 환경에서 --mode cuso 전체 256 후보를 먼저 실행한다.
2. 실패하면 cuso 식의 변수 선도항 정규화, x/y grouping, gap bit 처리 방식을 조정한다.
3. 백업으로 kionactf/coppersmith 의 multivariate linear Herrmann-May 구현에 같은 식을 넣어 실행한다.
4. 그래도 실패하면 SAT+Coppersmith hybrid 로 전환한다. SAT 는 임의 위치 bit constraint 를 잘 활용하고, Coppersmith 는 대수 구조를 활용하므로 조합이 자연스럽다.

9. 참고 자료

- Herrmann and May, "Solving Linear Equations Modulo Divisors: On Factoring Given Any Bits", ASIACRYPT 2008: https://link.springer.com/chapter/10.1007/978-3-540-89255-7_25
- Ajani and Bright, "SAT and Lattice Reduction for Integer Factorization", ISSAC 2024: <https://arxiv.org/abs/2406.20071>
- cuso EUROCRYPT 2025 artifact, "Solving Multivariate Coppersmith Problems": <https://artifacts.iacr.org/eurocrypt/2025/a13/readme.html>
- kionactf/coppersmith: <https://github.com/kionactf/coppersmith>
- jvdsn/crypto-attacks: <https://github.com/jvdsn/crypto-attacks>

10. 현재 상태에 대한 정직한 메모

이 패키지는 완성된 풀이가 아니라 외부 강한 환경에서 계속 실행하기 위한 작업 에셋이다. 현재 컨테이너에서는 Sage/cuso/flatter 부재와 fpylll 성능 한계 때문에 최종 p, q, 평문을 얻지 못했다. 문제 상수, 비트 구조, 공격식, 후보 스캔 방식, 복호화 루틴은 모두 재현 가능하게 정리되어 있다.